

A Unified Approach to Bulk-Synchronous Parallel Observability Using Relational Dataflows

by
Ankush Jain

A DISSERTATION PROPOSAL
Submitted in partial fulfillment of
the requirements for the degree of

DOCTOR OF PHILOSOPHY
Electrical and Computer Engineering
Carnegie Mellon University

Committee:
Prof. George Amvrosiadis (Chair)
Prof. Greg Ganger
Prof. Andy Pavlo
Prof. Vyas Sekar
Dr. Brad Settlemyer

CARNEGIE MELLON UNIVERSITY
Pittsburgh, Pennsylvania
August 2025

Contents

Contents	i
1 Introduction	2
1.1 Motivation: Efficiency Bottlenecks in BSP	2
1.2 Our Work: A Comprehensive Study of BSP Efficiency	3
1.3 Thesis Statement	5
1.4 Document Structure	5
2 Background: Efficiency Bottlenecks in BSP Systems	6
2.1 The High-Performance Computing Environment	6
2.2 Data Management Challenges	7
2.3 Compute Efficiency Challenges	7
3 CARP: Adaptive In-Situ Range Partitioning	8
3.1 Motivation: Efficient Ingestion for Range Queries	8
3.2 CARP: System Design	8
3.3 Key Results: Ingestion and Query Performance	9
3.4 Lessons	10
4 Lessons from Optimizing Adaptive Mesh Codes	11
4.1 Motivation: Computational Efficiency in AMR Codes	11
4.2 Challenges of Telemetry-Driven Placement	11
4.3 Contribution 1: A Diagnostic Workflow	11
4.4 Contribution 2: The CPLX Placement Policy	12
4.5 Key Results	13
4.6 Lessons	13
5 ORCA: A Unified Framework for Declarative Observability	14
5.1 Motivation: Programmable and Real-time Cluster Observability	14
5.2 Background: The Observability Gap in BSP Systems	15
5.3 ORCA Architecture	15
5.4 Potential Usecases and Impact	16
6 Thesis Completion Roadmap	18
6.1 Current State	18
6.2 Evaluation Roadmap	18
6.3 Timeline	19
References	20

Abstract

The Bulk Synchronous Parallel (BSP) paradigm powers our most critical computational tools, from large-scale scientific simulations to distributed machine learning. As these applications scale, their efficiency has datacenter-level impact on both costs and time-to-discovery. Yet the tightly synchronized nature of BSP creates two distinct challenges: on the data path, bursts of checkpointed state strain storage and analytics infrastructure, while on the compute path, global synchronization amplifies sensitivity to execution variability. While traditionally addressed by separate tools, this thesis demonstrates that both problems can be unified by treating them as challenges of organizing and processing streaming data for insight.

We first address these data and compute path challenges through two separate efforts. On the data path, we built CARP to enable efficient, streaming ingestion of data from large scientific applications for range queries without expensive post-processing. CARP uses reconstructed histograms to adapt to dynamic and skewed key distributions, achieving query performance comparable to a full sort with $5\times$ faster ingestion. On the compute path, we tackled low cluster efficiency in adaptive mesh codes, typically attributed to load imbalance. Our work revealed that apparent load imbalance is often confounded by environmental noise and other execution artifacts, necessitating a custom query-driven diagnostic workflow to isolate root causes. This process enabled us to establish a predictable baseline and design a tunable placement policy (CPLX) that improves runtime by up to 21.6% over this baseline.

Both our efforts revealed that efficiency gains are predicated on a common input: summarized views of distributed application state. These views are not only necessary to surface bottlenecks, but also inform the selection, execution, and tuning of interventions. Existing observability tools force a choice between limited metrics that obscure underlying issues and comprehensive traces, which are expensive to collect and postprocess and unviable at scale. We propose a fundamental shift: instead of proactive collection of all event logs, treat them as fragments of the global view, and use declarative dataflows for programmable, on-demand collection and real-time aggregation of these fragments. ORCA enables these capabilities through an aggregation and control plane optimized for BSP telemetry structure, leveraging a tree-based overlay with SQL semantics and distributed query planning to minimize data movement. This replaces a fractured ecosystem of bespoke tools with a unified, query-driven framework for real-time observability across all layers and phases of distributed execution.

1 Introduction

1.1 Motivation: Efficiency Bottlenecks in BSP

The Bulk Synchronous Parallel (BSP) paradigm powers some of our most critical computational tools, from large-scale scientific simulations in astrophysics and climate science to distributed training of foundation models in machine learning. These workloads — consisting of large parallel computations interspersed with I/O and global synchronization — currently operate at the scale of tens of thousands of accelerator nodes [1–4] and continue to grow with time [5]. As these applications scale, their efficiency has datacenter-level impacts on both cost and time-to-discovery.

The tightly synchronized nature of BSP creates distinct challenges along the data and compute paths. On the data path, bursts of checkpointed state create massive I/O demands on the storage and analytics infrastructure [6–8]. On the compute path, frequent global synchronization amplifies the system’s sensitivity to execution variability [9–13]. Runtime artifacts that would be manageable in loosely coupled systems, such as minor OS-level interruptions or thermal throttling, become critical scaling barriers when every worker must wait for the slowest participant. We now examine each path in more detail.

Data Management. Scientific applications generate massive volumes of data, particularly during checkpointing, which places intense pressure on I/O systems. Reading and analyzing this data after the fact is costly and can significantly slow time-to-insight. To mitigate this, many workflows now employ in situ analytics [14, 15], which process data as it is produced to reduce storage overhead and improve interactivity. While promising, most in situ frameworks remain specialized: they are typically tailored for visualization [16–20] or fixed-function transformations [21], and offer limited flexibility for general-purpose data organization. Critically, support for efficient range queries — essential for analyzing continuous scientific variables — is still missing. As a result, users are often forced to perform expensive post-processing sorts [22–24], reintroducing the I / O bottlenecks that in situ techniques aim to eliminate.

Compute Efficiency. Achieving high utilization remains challenging for many real-world BSP workloads [25]. In adaptive mesh codes [26], for instance, workload characteristics evolve dynamically as the simulation tracks changing physics, making it difficult to model or predict execution behavior. Even basic questions about compute load distribution, placement sensitivity, and communication overheads often lack clear answers. These effects are further confounded by environmental variability such as hardware faults or system noise—factors that can only be disentangled through in-situ observation. Traditional reliance on fine-grained tracing [27, 28] complicates diagnosis: traces are large, opaque, and difficult to query, while profiling tools lack the granularity to expose short-lived slowdowns that affect global synchronization.

1.2 Our Work: A Comprehensive Study of BSP Efficiency

We initially investigated BSP inefficiencies along the data and compute paths through two separate efforts: one focused on ingestion for range queries, the other on placement policies for dynamic workloads. While seemingly distinct, both efforts reflected specific instances of a broader pattern—establishing visibility into distributed cluster state as a precursor to context-aware interventions. This realization led to the design of ORCA, a programmable substrate for observability and control in production environments. The following sections describe each effort in turn.

1.2.1 Data Path: Streaming Data Reorganization

Motivation. Scientific analyses on continuous physical properties such as wavefronts or energy bands require efficient range queries [29]. Existing workflows typically achieve this by using a bulk sort [22–24] to reorder data after it is written, at the cost of multiple additional passes over the data. While in-situ ingestion systems exist for hash-partitioned layouts [30], they are unsuitable for range queries as hash-partitioning destroys key locality in the on-disk layout. Our goal was to build a completely in-situ ingestion mechanism for range queries, that did not require any reordering of on-disk data.

Challenges. While streaming ingestion for range queries can be done using range partitioning, doing it in a single pass requires perfect knowledge of the key distribution to generate balanced partitions. We found that to be impractical, as real scientific datasets are often highly skewed and evolve over time. Even well-chosen static partitions become imbalanced, resulting in write bottlenecks and degraded query performance. To be practical, a system must ingest data with arbitrary, unknown, and dynamic distributions, adapt to maintain balanced partitions on the fly, and scale across thousands of cores.

Our Contribution. We address these challenges with CARP [31] (*Continuous Adaptive Range Partitioner*), an adaptive partitioner that handles arbitrary key distributions and produces a partially ordered layout in a single streaming pass. CARP provides query latencies comparable to those of a full sort without any ingestion overhead, making it 5× faster than prior work.

1.2.2 Compute Path: Understanding and Mitigating Load Imbalance

Motivation. Poor utilization in adaptive mesh codes is a well-known problem, typically characterized as load imbalance [25, 26, 32]. We aimed to design placement policies that would incorporate runtime telemetry into placement decisions—not only to mitigate load imbalance, but also to balance other properties that we found to materially affect application runtime.

Challenges. Our efforts to collect and interpret telemetry, and create models of code performance that could be exploited in a placement policy encountered several persistent challenges.

1. Raw telemetry measurements did not correlate with expected behavior, forcing deep investigation of cross-stack performance anomalies, and revealing issues with hardware, network

stack parameters, operation ordering etc. These constituted runtime noise that had to be eliminated before telemetry and placement effects could be reliably observed.

2. Identifying, isolating, and tracing transient performance issues in a large parallel code required collecting and analyzing a large volume of telemetry. Existing performance tools were unhelpful: profiling tools did not provide the required granularity, while traces were slow and not analyzable.
3. Computing placements efficiently remained difficult, as even the simplest variant of the underlying problem is NP-hard and must be solved within tight timing constraints—under 50 ms per solution for our selected codes.

Our Contributions. We developed a diagnostic workflow and conducted an empirical tuning study to disentangle the effects of load imbalance from other runtime bottlenecks. These insights guided the design of CPLX, a hybrid placement policy with tunable control over two key tradeoffs: compute load balance and communication locality. By balancing both objectives, CPLX achieves up to 21.6% runtime improvement over tuned baselines [33].

1.2.3 ORCA: Transferring Lessons Learned to General-Purpose Observability

Motivation. Our work on both CARP and adaptive mesh codes revealed a common prerequisite for efficiency: the ability to construct a global view of distributed state to guide intervention. For CARP, this meant reconstructing the global key distribution to drive adaptive data reorganization. For mesh codes, it meant building a coherent performance picture to distinguish algorithmic bottlenecks from system noise. This visibility currently relies on tracing [27, 34], where prohibitive collection and analysis costs make it unviable at scale. Our goal was to build a general-purpose substrate that could efficiently materialize system views using on-demand and in-situ aggregation of telemetry.

Our Approach. ORCA (*Observability with Real-time Control and Aggregation*) is a general-purpose dataflow framework for cluster observability. Users define telemetry and control requirements as SQL queries over Arrow [35]-formatted data streams. These queries are compiled into distributed execution plans over a tree-structured overlay of aggregators and executed over per-timestep telemetry. Data is collected and analyzed only in response to active queries, enabling analytical costs to scale with analytical requirements. An asynchronous but timestep-consistent control path provides primitives to act on the basis of the feedback.

Challenges. To the best of our knowledge, ORCA is the first query-driven real-time observability system for tightly-coupled parallel applications. This requires solving challenges across the stack: from high-throughput, low-overhead data collection and scalable in-situ query processing, to the design of a control path that provides timestep-consistent semantics for an asynchronous human operator.

1.3 Thesis Statement

This thesis argues that persistent efficiency bottlenecks in *Bulk Synchronous Parallel* (BSP) systems are fundamentally visibility problems, stemming from an inability to reconstruct global views from distributed local state in real time, and proposes a programmable, query-driven observability and control plane to fill this gap.

1.3.1 Potential Impact

Immediate Impact. Application-level observability becomes a first-class capability. ORCA replaces brittle traces and low-resolution profilers with a lightweight, always-on, query-driven system that supports real-time diagnosis and runtime tuning. Developers can rapidly inspect distributed behavior, toggle probes, and react to anomalies without modifying application code or halting execution.

Longer-Term Impact. ORCA evolves into a flexible substrate for in-situ analytics and adaptive control. Its open architecture supports programmable analysis pipelines, integration of diverse telemetry sources, and runtime composition of custom indexing and aggregation strategies. Over time, it can subsume visualization, introspection, and control workflows into a unified framework that prioritizes insight per unit of data moved, enabling principled trade-offs between data fidelity, performance, and resource usage.

1.4 Document Structure

The rest of this document is organized as follows.

- Chapter 2 provides background on the HPC environment, application types, and known efficiency challenges.
- Chapter 3 describes CARP, an adaptive range partitioner.
- Chapter 4 describes our work on tuning and load-balancing adaptive mesh codes.
- Chapter 5 describes ORCA, a general-purpose observability and control framework.
- Chapter 6 outlines the remaining work required to complete the thesis.

2 Background: Efficiency Bottlenecks in BSP Systems

We first review key concepts and challenges in the bulk-synchronous parallel (BSP) paradigm, beginning with an overview of the computing environment (§2.1), followed by a review of data management (§2.2) and compute efficiency challenges (§2.3). While we primarily focus on scientific computing, we also discuss distributed machine learning training workloads, as their shared bulk-synchronous characteristics create similar challenges and inform our later work.

2.1 The High-Performance Computing Environment

System Architecture. Large-scale HPC clusters typically consist of $O(10,000)$ nodes [36] connected by a high-performance fabric [37–39]. Each node typically consists of CPUs and typically multiple GPUs, which are increasingly important for scientific computing [40, 41]. Storage is traditionally provided by parallel filesystems [42] but object stores [43] and richer commercial offerings [44, 45] are increasingly common [46].

Applications. Scientific applications target fundamental questions across domains such as cosmology and climate science, problems that require simulating complex physical phenomena over extended spatial and temporal scales. These applications can occupy the majority of a supercomputer’s resources for months at a time [1, 2, 47]. From a systems perspective, they can be simplified into two common archetypes: *particle codes* [48] and *mesh codes* [26]. Particle codes track the state and interactions of billions individual discrete entities as they move through the simulation domain. Mesh codes discretize the spatial domain into grids and compute the evolution of properties of interest at each grid point.

The BSP Model. The BSP model consists of alternating compute and synchronization phases. Execution begins with decomposition of the problem into independent subproblems assigned to workers, and proceeds as discrete timesteps. Each timestep consists of a compute phase followed by a global synchronization phase, where synchronous collectives [49, 50] ensure all ranks advance together. Every few timesteps, a checkpoint is triggered for both fault tolerance and offline analysis. This tight coupling is the root of cluster efficiency problems: synchronous checkpoints create coordinated I/O bursts that stress the storage system, while global collectives make overall performance acutely sensitive to the slowest worker. Growth in cluster sizes leads to increased checkpointing frequency, as the increased failure probability makes forward progress more valuable [6].

Distributed ML Training. Distributed ML training represents a similar and increasingly important cluster workload, occupying tens of thousands of GPUs and running for months [3–5]. The cluster architectures are comparable, though ML clusters often add a *frontend* Ethernet network for general datacenter connectivity [51]. While the parallelization strategies differ—ML employs a mix of data, tensor, and pipeline parallelism—the execution model still relies on the same core BSP primitives of synchronous collectives [51] and periodic checkpointing [7].

2.2 Data Management Challenges

Checkpointing creates I/O-bound bottlenecks in BSP systems, as coordinated writes across thousands of ranks stress parallel filesystems and consume significant runtime [52]. Traditional workflows then require scientists to re-read and analyze this data after simulation completion, consuming additional I/O bandwidth and computational resources while delaying scientific discovery. Ingesting data using conventional databases is not viable, as databases incur multiple on-disk writes for each application write [53] (*write amplification*), offering poor tradeoffs for ingestion-heavy workloads.

In-situ processing [15] analyzes data while it streams from application to storage, reducing both I/O overhead and time-to-insight. This approach is employed for multiple scientific use cases ranging from visualization [16, 17, 20, 54] to indexing [21, 55]. In-situ reorganization using hash-partitioning and shuffling has been demonstrated to be viable at HPC scale [30]. However, unlike databases that provide general query interfaces over composable indexes, existing in-situ systems are purpose-built for specific use cases and lack programmability. A critical gap remains in range query support, essential for analytics on continuous physical attributes but absent from existing in-situ frameworks.

2.3 Compute Efficiency Challenges

Cluster utilization challenges persist in BSP systems: a study of exported traces found more than 60% of runtime spent in MPI [49] operations (communication and synchronization), even at modest scales. We now provide additional context on performance tooling and workload characterization bottlenecks that contribute to these challenges.

Performance Analysis Tools. While real time cluster-level monitoring is common [56], application-level performance analyses are largely post-mortem [27, 34, 57, 58]. Two main approaches are tracing and profiling. Tracing generates fine-grained event logs for each worker (*rank*), which are then aggregated into a composite view via postprocessing. The volume of generated trace data presents an analytics challenge, delaying insight [59], and remains unviable at the largest scales [60]. In comparison, profiling generates phase-wise summaries of execution for each rank. While far more scalable, this obscures fine-grained interactions that degrade tightly-coupled BSP execution.

Workload Characterization Bottlenecks. While efficiency challenges are known and attributed to load imbalance [25, 26, 32], deeper understanding remains elusive, partly because of visibility bottlenecks discussed above. Characterization of workload properties such as compute kernel distributions, communication bottlenecks, placement and load-balancing requirements remain poorly understood. This is further exacerbated by the separation of analyses from data collection, as the latter are undertaken by separate groups without access to the underlying hardware that generated the data [61]. As a result, distinguishing workload characteristics from environment artifacts becomes even more difficult.

3 CARP: Adaptive In-Situ Range Partitioning

3.1 Motivation: Efficient Ingestion for Range Queries

Range queries are an essential component of a complete analysis toolkit. Analyses on scientific attributes of interest, such as wavefronts and energy bands, naturally map to range queries [29] because of the continuous nature of this data. However, supporting efficient range-based analyses remains expensive. Efficient range queries require an ordered data layout, which is created using bulk post-processing sorts after the application has finished executing. These sorts require multiple passes over the data — $2\times$ for in-memory [22, 23] sorts and $4\times$ for out-of-core [24] sorts — diverting finite I/O bandwidth, slowing down ingestion, and delaying the time to insight. As an alternative to post-processing, in situ indexing techniques have gained traction [14]. These techniques process data while in transit from application to storage, reducing the need for additional passes. DeltaFS [30] reorganizes data using hash-based partitioning, and has been demonstrated to scale up to 131K ranks. However, partitioning by key hash destroys the ordering needed for efficient range queries.

CARP [31] is a single-pass streaming in situ range partitioner for scientific data. It is designed to ingest at the maximum throughput permitted by the storage system by never reordering previously written data, while still enabling subsecond range queries. The fundamental challenge is maintaining balanced partitions for streams with skewed and dynamic key distributions. Our analysis demonstrates that scientific key distributions are highly skewed and evolve significantly as simulations progress, rendering any static partitioning approach ineffective (Fig. 1). CARP makes no assumptions about the workload key distribution, and instead discovers and adapts to it at runtime.

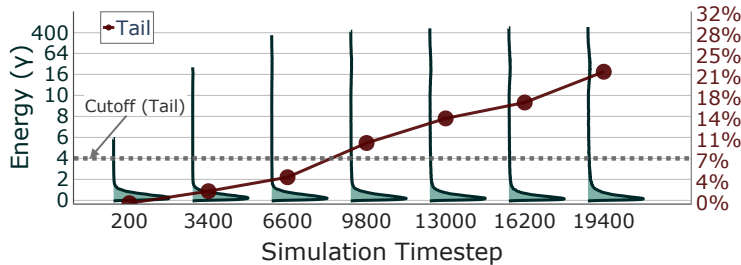


Figure 1: Energy distributions across 7 timesteps of a particle simulation (VPIC [29]). Intra-timestep distributions are highly skewed, with a long tail that grows significantly over time.

3.2 CARP: System Design

CARP is implemented as an in situ shuffling pipeline built around an all-to-all shuffler, *3-hop*, from the DeltaFS [30] project (see Figure 2). Each application rank acts as both a shuffle sender and receiver. In the common case, all ranks are assigned disjoint ranges, and shuffling simply forwards data to the rank responsible for the partition containing the indexed attribute value. On

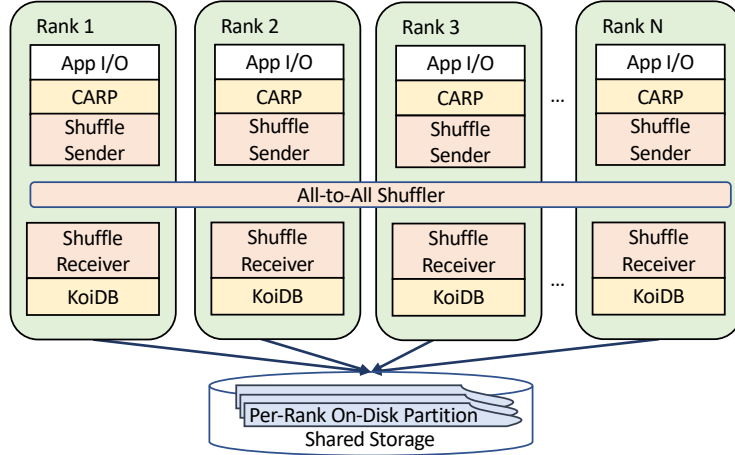


Figure 2: Example of an N -rank application using CARP. Writes are intercepted and partitioned via an all-to-all shuffler. Each rank is both a shuffle sender and a receiver. Partitioned data collected by receives is stored by a lock storage backend (KoiDB).

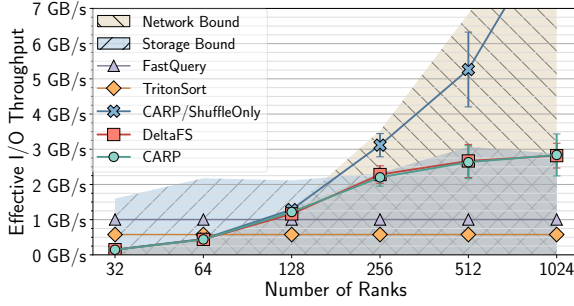
the receiver side, each rank accumulates writes into an embedded storage backend called *KoiDB*, which periodically flushes to a per-rank append-only log on the parallel filesystem.

CARP augments this shuffling pipeline with a primitive called *renegotiation*, which discovers the current global partitioning and computes new partition assignments. Any rank may invoke renegotiation. When triggered, each rank summarizes its local key distribution and sends it to the lead rank (rank 0), which merges the summaries to reconstruct the global key distribution. The lead rank then divides the global keyspace into partitions with equal area under the curve and broadcasts the new boundaries to all ranks. When partitions are updated, ranks flush their KoiDB instance and resume shuffling. For scalability, renegotiation is implemented as a tree-based reduction protocol.

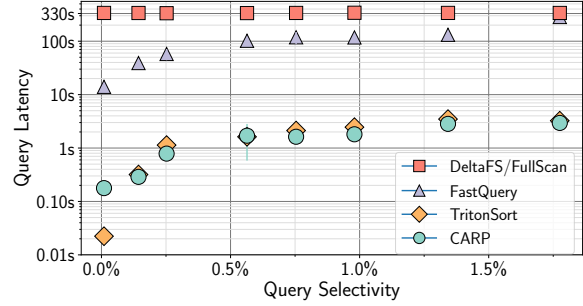
In addition to renegotiation, CARP includes mechanisms to trigger the process, bootstrap the distribution, and buffer data for unassigned ranges via the *out-of-bounds* (OOB) buffer. Filling the OOB buffer is one trigger for renegotiation, and its contents are factored into the partition computation. The KoiDB backend performs additional reprocessing to maintain partition quality despite out-of-order receives and rapidly changing boundaries, before periodically appending data to the per-rank log.

3.3 Key Results: Ingestion and Query Performance

Ingestion Performance. Fig. 3a shows the ingestion performance of CARP compared to TritonSort [24] (an out-of-core sort), FastQuery [29] (an auxiliary bitmap index), and DeltaFS [30] (in-situ hash partitioning). Being embedded in-situ approaches, CARP and DeltaFS are evaluated with different application ranks, while TritonSort and FastQuery are run as postprocessing stages



(a) Ingestion Throughput vs Application Scale



(b) Query Latency vs Selectivity

Figure 3: (Left) CARP incurs no overhead over unpartitioned I/O and is 3–5 $\times$ faster than post-processing. (Right) CARP matches the query latency of TritonSort’s fully-sorted data layout and outperforms FastQuery by 100 $\times$.

directly on the storage nodes. CARP achieves the maximum throughput permitted by the storage and network bounds. CARP has no measurable overhead on the ingestion path. At 1024 ranks, it is 2.8 $\times$ faster than FastQuery and 4.9 $\times$ faster than TritonSort.

Query Performance. Fig. 3b shows the query performance of CARP compared to FastQuery, TritonSort, and a complete sequential scan (DeltaFS/FullScan) for 8 queries with different selectivities. For all but the most selective query, CARP queries are as fast as a full sort and 1-2 orders of magnitude faster than FastQuery. Being an auxiliary index, FastQuery is forced to perform millions of random I/Os, resulting in relatively poor query performance approaching the cost of a complete sequential scan. The gap between CARP and TritonSort for the most selective query is explained by CARP being a partitioner, and is thus forced to read a complete partition for highly selective queries. Given the small absolute value (25ms) for such queries, the gap is not meaningful for analytics queries.

3.4 Lessons

CARP demonstrates that a greedy, single-pass approach can produce a partially ordered layout that supports efficient range queries without compromising ingestion performance. By approximating a clustered index over streaming data, it narrows query scope to a small number of partitions, reducing retrieval cost. The performance gap between CARP and auxiliary index-based approaches highlights the importance of physical partitioning as a first-class indexing mechanism. More broadly, CARP shows that scientific data systems benefit from the same design principles as databases, yet they currently lack the crucial ability to compose different indexing strategies at query time. The practical realization of these benefits requires adopting interoperable data formats and query engines that can leverage the available indexing and partitioning schemes.

4 Lessons from Optimizing Adaptive Mesh Codes

4.1 Motivation: Computational Efficiency in AMR Codes

Adaptive Mesh Refinement (AMR) [26, 62, 63] is a widely used technique for efficiently simulating physical systems that exhibit large variations in spatial or temporal scales, such as astrophysical phenomena, fluid dynamics, and climate modeling. AMR codes dynamically refine and coarsen the computational grid to concentrate resources in regions of interest, resulting in highly irregular and rapidly evolving mesh structures. This approach enables fine-grained resolution where needed, but creates significant computational challenges: the workload becomes highly imbalanced, dependencies between tasks grow complex, and the dynamic nature of the mesh makes it difficult to achieve efficient parallel execution and balanced load across compute resources [25, 26, 32].

Our goal was to develop placement policies that leveraged runtime telemetry to improve placement and efficiency. Access to our research cluster enabled us to experiment with fine-grained telemetry and placement under full-stack administrative control, providing the ability to analyze and intervene at all levels of the software and hardware stack.

4.2 Challenges of Telemetry-Driven Placement

Developing telemetry-driven placement policies in AMR codes exposed several key challenges:

- **Establishing measurement fidelity.** Initial telemetry was often unreliable and did not match intuition (Fig. 4). Only by using full-stack access and low-level debugging could we identify and correct sources of error across hardware, drivers, MPI, and the application.
- **Scaling analysis for high-volume telemetry.** Analyzing per-rank, per-timestep telemetry required moving beyond standard profiling and tracing tools to query-driven workflows that could surface root causes from massive data.
- **Low-overhead, effective placement.** Even with reliable telemetry, incorporating it into placement imposed strict timing constraints and required new algorithms to balance objectives like load and locality in dynamic, NP-hard settings.

4.3 Contribution 1: A Diagnostic Workflow

The process of developing placement policies required understanding what our telemetry was capturing. Initially, our telemetry showed no correlation between expected workload drivers and observed performance (Figure 4b). Deeper investigation revealed cross-stack artifacts: fail-slow hardware, system tuning issues, and fabric-level problems that had to be systematically mitigated (Fig. 4a). Only after this full-stack debugging and tuning process could we obtain reliable telemetry that correlated with expected application behavior.

While our full-stack access enabled these investigations, we found identifying and isolating these transient artifacts to be difficult. We initially used standard profiling and tracing tools [57], but found the behavior of our applications to necessitate structured and targeted telemetry collection for

analytics. Profiling often obscured visibility into transient artifacts, while tracing resulted in large volumes of analytically intractable data. We organically evolved a workflow for targeted collection of queryable telemetry, starting from CSV exports analyzed with pandas [64], and culminating in ClickHouse [65] and SQL queries. This custom approach enabled correlation of performance data across ranks and system layers, allowing us to isolate the true sources of performance anomalies.

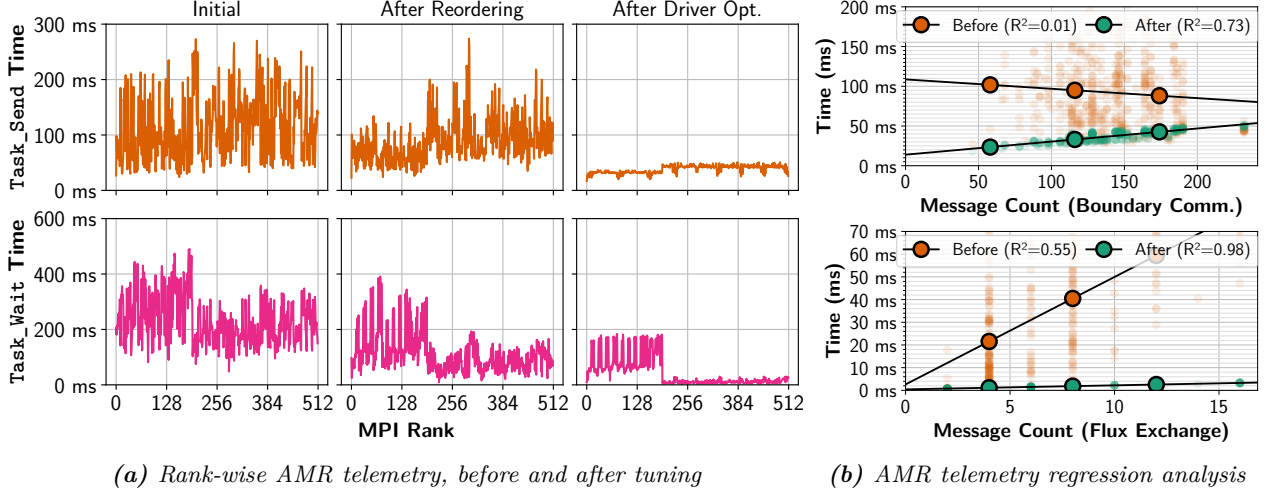


Figure 4: (Left) Fine-grained per-rank telemetry from communication functions showing significant variability, that progressively reduces with tuning. (Right) Regression analysis of communication telemetry showing poor initial correlation with work, that improves significantly with tuning.

4.4 Contribution 2: The CPLX Placement Policy

With reliable telemetry established, we turned to the challenge of developing more effective placement policies. Placement strategies for AMR codes have traditionally prioritized maximal preservation of communication locality [66–70], using tools like *Space-Filling Curves* [71] (SFCs) to co-locate spatially adjacent grid blocks. However, we recognized that for computationally variable workloads, an exclusive focus on locality is fundamentally at odds with achieving good load balance. This led us to reframe placement as not about maximizing a single objective, but about explicitly managing the trade-off between communication and computation to minimize overall runtime.

To navigate this trade-off, we developed CPLX, a policy that provides predictable control over locality and load balance through a single tunable parameter, X . CPLX operates as a hybrid of two underlying policies: a locality-preserving dynamic programming approach (CDP) developed by us and the load-balancing-focused *Longest Processing Time* (LPT) heuristic [72]. The algorithm first computes a locality-aware placement using CDP. It then identifies the $X\%$ most imbalanced ranks (both overloaded and underloaded) and rebalances *only* their assigned work using LPT, leaving the rest of the placement untouched. This mechanism allows CPLX to smoothly interpolate between a fully locality-preserving policy ($X = 0$) and a pure load-balancing policy ($X = 100$), enabling empirical tuning of the trade-off.

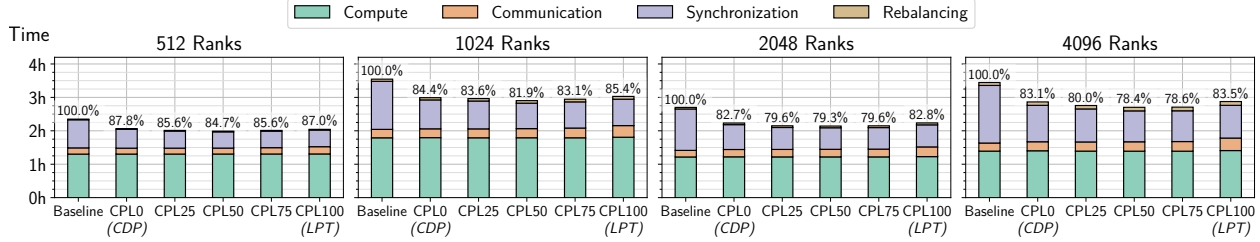


Figure 5: End-to-end AMR code runtime as a function of policy at four scales (512–4096 ranks), classified into four phases. CPLX achieves up to 21.6% runtime reduction over already-optimized baselines. Total runtime follows a U-shaped curve as X increases, with best performance at intermediate values, from balancing the constituent tradeoffs. Impact of CPLX increases with scale.

4.5 Key Results

We evaluated our placement policies on our in-house research cluster using the Phoebus [73] code built on the Parthenon AMR framework [62], at scales ranging from 512 to 4096 ranks. We chose the Sedov blast wave [74] problem as our workload, as its dynamic refinement patterns make it a challenging test case for placement policies. All comparisons are against the tuned baseline using standard SFC-based placement.

As shown in Figure 5, CPLX achieves up to a 21.6% runtime reduction over the optimized baseline. Overall performance follows a U-shaped curve as X is varied, with the best results achieved at intermediate values. At $X = 0$, CPLX is locality maximizing at the expense of suboptimal load balance, while at $X = 100$, CPLX completely disregards locality to maximize load-balance. Minimum runtime is achieved at intermediate X values by balancing the constituent trade-offs. The impact of CPLX and managing the trade-off both increase with scale.

4.6 Lessons

Elevated synchronization time in AMR codes is commonly ascribed to algorithmic load imbalance. While execution variability is a known issue [9–11, 13, 51, 75, 76], it is often overlooked in performance analyses, which are typically conducted offline and disconnected from runtime context [25]. Our work shows that accurate diagnosis requires holistic, in situ performance analysis to isolate the true sources of variability. We find that algorithmic techniques can improve placement, but they also expose tunable tradeoffs—such as load balance versus communication locality—whose optimal configuration also benefits from real-time visibility into application behavior. Our organically evolved pipeline demonstrates the value of structured, queryable telemetry, and motivates the realization of real-time observability capabilities tailored to the needs of tightly-coupled parallel applications.

5 ORCA: A Unified Framework for Declarative Observability

5.1 Motivation: Programmable and Real-time Cluster Observability

Both CARP and our work on AMR placement reveal a persistent and recurring need to reconstruct cluster state to synthesize views that enable better decision-making. CARP automatically uses this reconstructed state to inform online data reorganization, and our AMR work reveals the urgent need for such capability on the compute path, where disambiguating between load imbalance, system variability, and execution strategies is essential for computational efficiency.

Although the telemetry workflow evolved during our AMR placement work demonstrated the importance of structured and queryable fine-grained telemetry, the pipeline itself remained ad hoc and limited. We derive three requirements for practical observability capabilities for production clusters from our experience.

- **Real-time and Coordinated Analytics.** Insights should be available in real time, with minimal application overhead, while also allowing the option to persist detailed telemetry for offline analysis. This requires coordinated, timestep-wise telemetry collection from all ranks.
- **Complete Programmability.** Effective observability requires supporting analytics over arbitrary data sources across the application stack, with varying schemas and aggregation requirements. We found fixed aggregations or rigid schemas to quickly become limiting as requirements evolve—programmability is essential for sustained utility.
- **Interactivity.** Observability workflows vary significantly across the lifetime of a long-running BSP application, and an effective framework must support the full span of these workflows while incurring costs commensurate with the analysis requirement. Lightweight monitoring may suffice in the common case, but when performance anomalies arise, detailed telemetry becomes essential. The framework must enable dynamic transitions between these modes without fixed overhead penalties.

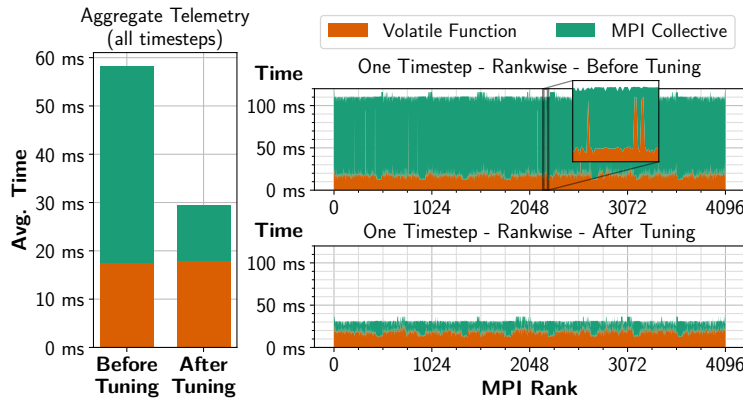


Figure 6: An example of a BSP performance problem from a real AMR code, simplified as a volatile function followed by global synchronization. (Left) Aggregated telemetry shows elevated average synchronization times, but does not reveal the problem structure. (Right) Fine-grained telemetry shows random spikes in the volatile function delaying synchronization. Tuning the volatile function does not affect its average time, but leads to a 50% reduction in their combined time.

5.2 Background: The Observability Gap in BSP Systems

Fig. 7 compares different observability approaches for BSP telemetry, modeled as a two-dimensional grid of ranks and timesteps. Among existing approaches [27], comprehensive tracing [34, 57, 77] is the only method that preserves the fine-grained telemetry needed to diagnose complex performance issues in large-scale BSP systems (see Fig. 6). However, the high application overhead, unstructured and analytics-unfriendly output [28, 59], and delayed feedback from post-processing make tracing impractical for production use. As clusters and their resource footprints continue to grow, these limitations become more acute, necessitating a ground-up rethinking of observability architectures to keep pace with modern system demands.

Profiling [57] and sampling [56] tools, while lower overhead, impose arbitrary fixed collection and aggregation patterns that obscure the structure of underlying problems and mask important temporal or spatial correlations. Time Series Databases (TSDBs) [78, 79] — widely used for metrics in distributed services — are not a direct fit for scalable BSP observability, as they are not optimized for continuous time-windowed aggregations rather than discrete timestep-synchronized analytical queries over BSP telemetry. As a result, cluster efficiency challenges persist not due to algorithmic bottlenecks, but because of persistent limitations in what can be observed, analyzed, and acted on in real time.

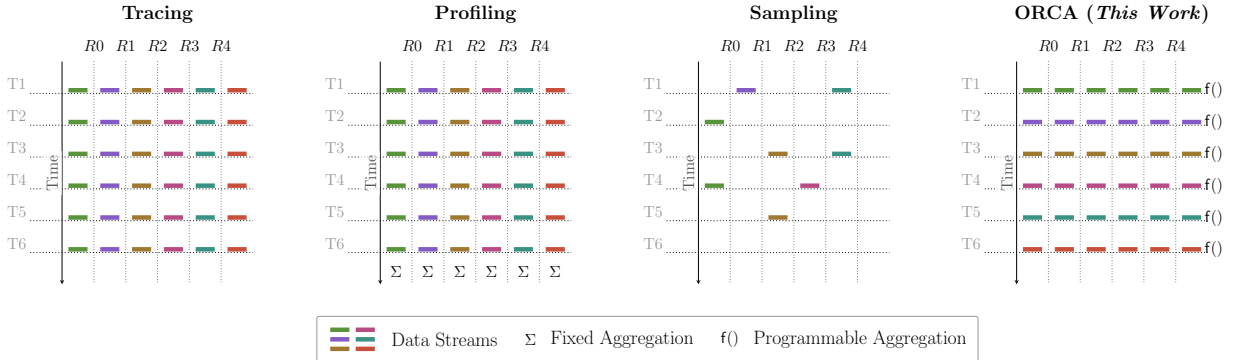


Figure 7: A comparison of different observability approaches for BSP telemetry, modeled as a two-dimensional grid of ranks and timesteps. Tracing captures all event data, but writes it as separate per-rank streams to storage, posing analytical challenges. Profiling and sampling offer arbitrary fixed reductions of data, obscuring insight. ORCA enables coordinated collection of timestep-wise telemetry from all ranks, with programmable in-situ aggregation.

5.3 ORCA Architecture

ORCA is designed to address the challenges of scalable, real-time observability in large parallel systems via programmable aggregation and control paths. The framework enables users to express analytic intent as SQL over telemetry. Unlike tracing, which collects and stores all event data, ORCA leverages distributed query planning to transfer only the data necessary to materialize each requested view. The entire pipeline, from collection to aggregation to storage, is optimized

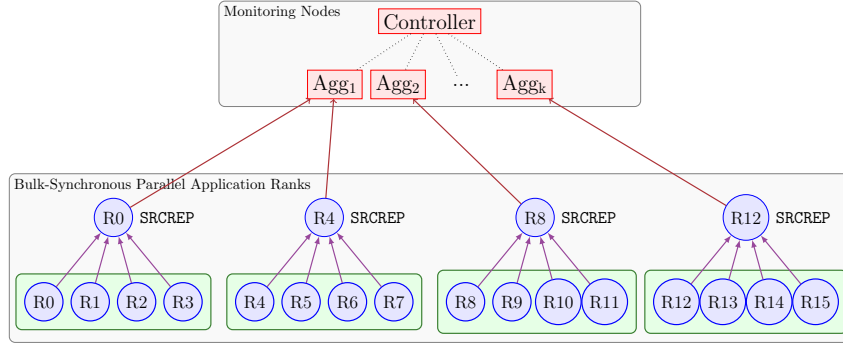


Figure 8: An example of an ORCA tree-based overlay topology, with 16 MPI ranks across 4 nodes connecting to a controller via a hierarchy of aggregators, and optionally a tier of local lead ranks. Application ranks form the synchronous domain while ORCA executes asynchronously.

to preserve the BSP structure of the data. Its control semantics enable dynamic transitions between different views at runtime, supporting interactive, low-overhead workflows. Below, we briefly describe the core components of the ORCA architecture.

- **Tree-Based Overlay Network (TBON):** As shown in Fig. 8, ORCA organizes application ranks into a logical tree-based overlay with a controller at the root, a tier of aggregators, optionally a tier of lead ranks (aggregating within node groups), and application ranks as leaves. Aggregators handle data collection from a subset of ranks, using RDMA for data transfer when available.
- **Distributed Query Planning:** Telemetry is represented in Apache Arrow columnar format, and the overlay implements distributed query planning using Apache Datafusion. Projection, filtering, and partial aggregation are pushed down to lower tiers, so only necessary data is transferred up the tree.
- **Control Semantics.** ORCA implements a control path using a novel variant of two-phase commit, called *timestep-linked two-phase commit* (TS2PC). TS2PC is an asynchronous protocol that enables control actions to be applied consistently at a specified timestep across all ranks. This provides low-overhead replicated state machine semantics and supports dynamic observability workflows.

5.4 Potential Usecases and Impact

ORCA enables a range of use cases across high-performance computing and machine learning workflows, while providing a foundation for broader analytics and control in the future.

HPC Workloads. ORCA enables continuous, always-on observability for large-scale simulation codes. Unlike traditional workflows that require choosing between lightweight profiling and expensive tracing, ORCA supports a seamless spectrum: developers can inject detail dynamically and escalate from low-overhead monitoring to deep, on-demand root-cause analysis as needed. This

removes silos between telemetry collection modes and provides actionable insight without workflow disruption.

ML Workloads. Distributed ML workloads, because of their shared BSP characteristics, present similar performance and observability challenges as HPC codes [51]. For example, PyTorch [80] enables tracing each node using Kineto [34], but relies on post-processing to aggregate per-node telemetry into global views [58]. Limitations discussed in the context of HPC observability — data volume, delayed feedback, and lack of analytical capabilities — apply equally here. Beyond performance, recent ML research has emphasized the need for runtime observability over application state to enforce correctness invariants [81]. We find that ORCA naturally accommodates such use cases because of its focus on programmability, enabling a common substrate for multiple forms of observability.

Longer-Term Vision. ORCA’s software-based architecture — incorporating open standards such as Apache Arrow [35], Datafusion [82], and Mercury [83] over libfabric [84] — makes it naturally extensible. Many scientific analyses depend on multidimensional and unstructured meshes, which are not straightforward to represent in the relational model of standard SQL. Prior work [85–88] provides clear paths for extending SQL to cover these data types, and ORCA is amenable to absorbing these capabilities, and enabling in-situ visualization workflows within the same pipeline. Previous work has also demonstrated the utility of accelerators for in-situ analysis on Arrow data [89, 90]. ORCA makes realizing these capabilities practical by exposing accelerators and smart network resources as nodes within its dataflow graph. This enables a unified analytics and control fabric for scientific and data-intensive computing ecosystems to emerge organically over time.

6 Thesis Completion Roadmap

ORCA is under advanced stages of development. We first briefly describe the current state of the system, and then outline the remaining work required to complete the thesis.

6.1 Current State

Completed Developmental Work. Most core components of ORCA have been implemented. Completed components include overlay bootstrapping with multiple aggregators and topologies, telemetry collection and forwarding mechanisms, with per-timestep analytics semantics. Observability requirements are programmed using a domain-specific language (DSL) called OrcaFlow: the DSL enables defining transformations as SQL, applied in-situ while data is routed to configurable *sinks*. The TS2PC protocol for the control path has been implemented. End-to-end telemetry visualization is available via a DuckDB [91] and a FlightSQL [92] server embedded into the controller, streamed to Grafana [93]. ORCA reuses the same infrastructure to export internal metrics for realtime visualization and performance analysis. Observability requirements from recent machine learning literature [81] have been reviewed and incorporated in system semantics.

Validated Capabilities. A single-aggregator deployment of ORCA has been validated to be stable in a stress test with 128 MPI ranks loading the aggregator with 2–3 GB/s of telemetry. Analytics were performed locally on the aggregator, achieving near-line rate performance under ideal conditions on 40 Gbps Infiniband NICs. Stress testing surfaced stability issues, which were mitigated by tuning pipeline parameters and implementing flow control mechanisms. We found basic Arrow-based in-memory analytics to be much faster than our ingestion rate. The RDMA-based transport layer has been tuned and shown to be stable.

Remaining Developmental Work. The primary remaining component is distributed query planning to fragment the OrcaFlow pipeline across the overlay at operator-level granularity. Development of the query planner is completed and integration with the overlay is underway. Python bindings for Pytorch [80] integration are also under development for the planned experiments.

6.2 Evaluation Roadmap

The evaluation will demonstrate the practical benefits of ORCA’s relational approach in terms of query performance, system scalability, and end-to-end impact. The plan consists of the following experiments:

- **Microbenchmarks:** Compare query efficiency and analytical flexibility using on-disk Parquet [94] telemetry versus traditional trace formats. AMR-inspired queries [33] will serve as the HPC baseline, and Pytorch HTA [58] will be used as the ML baseline.
- **Synthetic stress tests:** Measure ORCA throughput and scalability as a function of aggregator count, and quantify application jitter as a function of data load.

- **End-to-end workload studies:** Measure runtime overhead and impact on scientific progress for a 4,096-rank AMR simulation [33] and a data-parallel ML training job, comparing with the overhead of full tracing. Evaluate ORCA in two configurations: a lightweight in-situ aggregation setup and a full configuration that persists all telemetry at tracing volume. The ML training job will be a data-parallel job running on a single node, and the AMR simulation will be run on a single node.

Additionally, an analysis of observability requirements identified from ML training literature, demonstrating its compatibility with ORCA semantics, is ongoing. This is not expected to have an experimental component.

6.3 Timeline

The thesis is expected to be completed by the end of 2025. The following timeline is proposed:

- July–Aug 2025: Complete distributed query planning and wrap up core development.
- Sept–Oct 2025: Evaluation, paper writing, and job search.
- Nov–Dec 2025: Complete and defend thesis.

References

- [1] Sambit Das et al. “Large-Scale Materials Modeling at Quantum Accuracy: Ab Initio Simulations of Quasicrystals and Interacting Extended Defects in Metallic Alloys”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2023. URL: <https://dl.acm.org/doi/10.1145/3581784.3627037>.
- [2] Ryan Stocks et al. “Breaking the Million-Electron and 1 EFLOP/s Barriers: Biomolecular-Scale Ab Initio Molecular Dynamics Using MP2 Potentials”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*. SC ’24. 2024. URL: <https://dl.acm.org/doi/10.1109/SC41406.2024.00015>.
- [3] Aaron Grattafiori et al. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783 [cs]. URL: <http://arxiv.org/abs/2407.21783>. Pre-published.
- [4] *Meta Trains Llama 4 on a 100,000+ H100 GPU Supercluster - NADDOD Blog*. URL: <https://www.naddod.com/blog/meta-trains-llama-4-on-a-100-000-h100-gpu-supercluster>.
- [5] Robi Rahman. *AI Training Cluster Sizes Increased by More than 20x since 2016*. Epoch AI. 2024. URL: <https://epoch.ai/data-insights/training-cluster-size>.
- [6] William M. Jones, John T. Daly, and Nathan DeBardeleben. “Application Monitoring and Checkpointing in HPC: Looking towards Exascale Systems”. In: *Proceedings of the 50th Annual Southeast Regional Conference*. 2012. URL: <https://dl.acm.org/doi/10.1145/2184512.2184574>.
- [7] Assaf Eisenman et al. “{Check-N-Run}: A Checkpointing System for Training Deep Learning Recommendation Models”. In: 2022. URL: <https://www.usenix.org/conference/nsdi22/presentation/eisenman>.
- [8] Xinyu Lian et al. *Universal Checkpointing: A Flexible and Efficient Distributed Checkpointing System for Large-Scale DNN Training with Reconfigurable Parallelis*. Version 3. 2025. arXiv: 2406.18820 [cs]. URL: <http://arxiv.org/abs/2406.18820>. Pre-published.
- [9] Torsten Hoefer, Timo Schneider, and Andrew Lumsdaine. “Characterizing the Influence of System Noise on Large-Scale Applications by Simulation”. In: *SC ’10: Proceedings of the 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis*. 2010. URL: <https://ieeexplore.ieee.org/document/5645455>.
- [10] Fabrizio Petrini, Darren J. Kerbyson, and Scott Pakin. “The Case of the Missing Supercomputer Performance: Achieving Optimal Performance on the 8,192 Processors of ASCI Q”. In: *Proceedings of the 2003 ACM/IEEE Conference on Supercomputing*. 2003. URL: <https://dl.acm.org/doi/10.1145/1048935.1050204>.
- [11] Orcun Yildiz et al. “On the Root Causes of Cross-Application I/O Interference in HPC Storage Systems”. In: *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 2016. URL: <https://ieeexplore.ieee.org/document/7516071>.
- [12] Brian Kocoloski and John Lange. “Varbench: An Experimental Framework to Measure and Characterize Performance Variability”. In: *Proceedings of the 47th International Conference on Parallel Processing*. 2018. URL: <https://dl.acm.org/doi/10.1145/3225058.3225125>.
- [13] Bilge Acun, Phil Miller, and Laxmikant V. Kale. “Variation Among Processors Under Turbo Boost in HPC Systems”. In: *Proceedings of the 2016 International Conference on Supercomputing*. 2016. URL: <https://dl.acm.org/doi/10.1145/2925426.2926289>.
- [14] A. C. Bauer et al. “In Situ Methods, Infrastructures, and Applications on High Performance Computing Platforms”. In: *Computer Graphics Forum* 35.3 (2016). URL: <https://onlinelibrary.wiley.com/doi/10.1111/cgf.12930>.
- [15] Hank Childs et al. “A Terminology for in Situ Visualization and Analysis Systems”. In: *The International Journal of High Performance Computing Applications* 34.6 (2020). URL: <https://doi.org/10.1177/1094342020935991>.
- [16] Matthew Larsen et al. “Ascent: A Flyweight In Situ Library for Exascale Simulations”. In: *In Situ Visualization for Computational Science*. Ed. by Hank Childs, Janine C. Bennett, and Christoph Garth. 2022.
- [17] Utkarsh Ayachit et al. “ParaView Catalyst: Enabling In Situ Data Analysis and Visualization”. In: *Proceedings of the First Workshop on In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization*. 2015. URL: <https://dl.acm.org/doi/10.1145/2828612.2828624>.

- [18] Seif Ibrahim et al. “Interactive In Situ Visualization and Analysis Using Ascent and Jupyter”. In: *Proceedings of the Workshop on In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization*. 2019. URL: <https://dl.acm.org/doi/10.1145/3364228.3364232>.
- [19] Pascal Grosset and James Ahrens. “Lightweight Interface for In Situ Analysis and Visualization of Particle Data”. In: *ISAV’21: In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization*. 2021. URL: <https://dl.acm.org/doi/10.1145/3490138.3490143>.
- [20] Matthew Larsen et al. “The ALPINE In Situ Infrastructure: Ascending from the Ashes of Strawman”. In: *Proceedings of the In Situ Infrastructures on Enabling Extreme-Scale Analysis and Visualization*. ISAV’17. 2017. URL: <https://dl.acm.org/doi/10.1145/3144769.3144778>.
- [21] Greg Eisenhauer et al. *Streaming Data in HPC Workflows Using ADIOS*. 2024. arXiv: 2410.00178 [cs]. URL: <http://arxiv.org/abs/2410.00178>. Pre-published.
- [22] Bin Dong, Surendra Byna, and Kesheng Wu. “SDS-sort: Scalable Dynamic Skew-Aware Parallel Sorting”. In: *HPDC 2016 - Proceedings of the 25th ACM International Symposium on High-Performance Parallel and Distributed Computing*. 2016.
- [23] Hari Sundar, Dhairya Malhotra, and George Biros. “HykSort: A New Variant of Hypercube Quicksort on Distributed Memory Architectures”. In: *Proceedings of the International Conference on Supercomputing*. 2013.
- [24] Alexander Rasmussen et al. “TritonSort: A Balanced Large-Scale Sorting System”. In: *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation*. NSDI’11. 2011.
- [25] Benjamin Klenk and Holger Fröning. “An Overview of MPI Characteristics of Exascale Proxy Applications”. In: *High Performance Computing: 32nd International Conference, ISC High Performance 2017, Frankfurt, Germany, June 18–22, 2017, Proceedings*. 2017. URL: https://doi.org/10.1007/978-3-319-58667-0_12.
- [26] Anshu Dubey et al. “A Survey of High Level Frameworks in Block-Structured Adaptive Mesh Refinement Packages”. In: *Journal of Parallel and Distributed Computing* 74.12 (2014). URL: <https://linkinghub.elsevier.com/retrieve/pii/S0743731514001178>.
- [27] Brian J. N. Wylie et al. “15+ Years of Joint Parallel Application Performance Analysis/Tools Training with Scalasca/Score-P and Paraver/Extrae Toolsets”. In: *Future Generation Computer Systems* 162 (2025). URL: <https://www.sciencedirect.com/science/article/pii/S0167739X24004187>.
- [28] Eschweiler Dominic et al. “Open Trace Format 2: The Next Generation of Scalable Trace Formats and Support Libraries”. In: *Advances in Parallel Computing*. 2012. URL: <https://www.medra.org/servlet/aliasResolver?alias=iospress&issn=0927-5452&volume=22&spage=481>.
- [29] Surendra Byna et al. “Parallel I/O, Analysis, and Visualization of a Trillion Particle Simulation”. In: *International Conference for High Performance Computing, Networking, Storage and Analysis, SC*. 2012.
- [30] Qing Zheng et al. “Scaling Embedded In-Situ Indexing with DeltaFS”. In: *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*. 2018. URL: <https://ieeexplore.ieee.org/document/8665820>.
- [31] Ankush Jain et al. “CARP: Range Query-Optimized Indexing for Streaming Data”. In: *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. 2024.
- [32] Carsten Burstedde et al. “Scalable Adaptive Mantle Convection Simulation on Petascale Supercomputers”. In: *2008 SC - International Conference for High Performance Computing, Networking, Storage and Analysis*. 2008. URL: <http://ieeexplore.ieee.org/document/5214248/>.
- [33] Ankush Jain et al. “Lessons from Profiling and Optimizing Placement in AMR Codes”. In: *2025 IEEE International Conference on Cluster Computing (CLUSTER)*. –Sept. 30, 2025.
- [34] *Pytorch/Kineto*. pytorch, 2025. URL: <https://github.com/pytorch/kineto>.
- [35] *Apache Arrow*. Apache Arrow. URL: <https://arrow.apache.org/>.
- [36] *TOP500 List - June 2025 — TOP500*. URL: <https://top500.org/lists/top500/list/2025/06/>.
- [37] Gregory F Pfister. “An Introduction to the Infiniband Architecture”. In: *High performance mass storage and parallel I/O* 42.617–632 (2001).
- [38] Duncan Roweth et al. “HPE Slingshot Launched into Network Space”. In: *Cray User Group* (2022).

- [39] J Metz. “Empowering AI Workloads in Ultra Ethernet Consortium”. In: *2024 IEEE Photonics Society Summer Topicals Meeting Series (SUM)*. 2024. URL: <https://ieeexplore.ieee.org/document/10614558/>.
- [40] Nathan DeBardeleben et al. “GPU Behavior on a Large HPC Cluster”. In: *Euro-Par 2013: Parallel Processing Workshops*. Ed. by Dieter an Mey et al. 2014.
- [41] Addison Snell and Laura Segervall. “HPC Application Support for GPU Computing”. In: (2017).
- [42] Peter J Braam and Philip Schwan. “Lustre: The Intergalactic File System”. In: *Ottawa Linux Symposium*. Vol. 8. 11. 2002.
- [43] Michael Hennecke. “Daos: A Scale-out High Performance Storage Stack for Storage Class Memory”. In: *Supercomputing frontiers* 40 (2020).
- [44] *Hammerspace Data Platform*. Hammerspace. 2024. URL: <https://hammerspace.com/hammerspace-global-data-platform/>.
- [45] *The VAST Platform White Paper*. URL: <https://www.vastdata.com>.
- [46] Glenn Lockwood. *LLM Training without a Parallel File System*. 2025. URL: <https://blog.glennklockwood.com/2025/02/llm-training-without-parallel-file.html>.
- [47] Gary Grider. “LANL Platform Planning and Update” (HPC User Forum 2023). 2023. URL: <https://hpcuserforum.com/wp-content/uploads/2023/09/Gary-Grider-LANL-Platform-Planning-and-Update.pdf>.
- [48] Robert Bird et al. “VPIC 2.0: Next Generation Particle-in-Cell Simulations”. In: *IEEE Transactions on Parallel and Distributed Systems* 33.4 (2022). URL: <https://ieeexplore.ieee.org/document/9444146/>.
- [49] “MPI: A Message-Passing Interface Standard”. In: ().
- [50] *NVIDIA Collective Communications Library (NCCL)*. NVIDIA Developer. URL: <https://developer.nvidia.com/nccl>.
- [51] Adithya Gangidi et al. “RDMA over Ethernet for Distributed Training at Meta Scale”. In: *Proceedings of the ACM SIGCOMM 2024 Conference*. 2024. URL: <https://dl.acm.org/doi/10.1145/3651890.3672233>.
- [52] Bing Xie et al. “Characterizing Output Bottlenecks in a Supercomputer”. In: *SC ’12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 2012. URL: <https://ieeexplore.ieee.org/document/6468446/>.
- [53] Pandian Raju et al. “PebblesDB: Building Key-Value Stores Using Fragmented Log-Structured Merge Trees”. In: *SOSP 2017 - Proceedings of the 26th ACM Symposium on Operating Systems Principles*. 2017.
- [54] Utkarsh Ayachit et al. “The SENSEI Generic In Situ Interface”. In: *2016 Second Workshop on In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization (ISAV)*. 2016. URL: <http://ieeexplore.ieee.org/document/7836400/>.
- [55] Ciprian Docan, Manish Parashar, and Scott Klasky. “DataSpaces: An Interaction and Coordination Framework for Coupled Simulation Workflows”. In: *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing*. HPDC ’10. 2010. URL: <https://dl.acm.org/doi/10.1145/1851476.1851481>.
- [56] Benjamin Schwaller. *Integrated System and Application Continuous Performance Monitoring and Analysis Capability (Final)*. SAND2021-11954, 1822583, 700207. 2021. URL: <https://www.osti.gov/servlets/purl/1822583/>.
- [57] Sameer S. Shende and Allen D. Malony. “The Tau Parallel Performance System”. In: *Int. J. High Perform. Comput. Appl.* 20.2 (2006). URL: <https://doi.org/10.1177/1094342006064482>.
- [58] *Holistic Trace Analysis — Holistic Trace Analysis 0.5.0 Documentation*. URL: <https://hta.readthedocs.io/en/latest/index.html>.
- [59] Hariharan Devarajan et al. “DFTracer: An Analysis-Friendly Data Flow Tracer for AI-Driven Workflows”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*. SC ’24. 2024. URL: <https://dl.acm.org/doi/10.1109/SC41406.2024.00023>.

- [60] Weihao Cui et al. *XPUTimer: Anomaly Diagnostics for Divergent LLM Training in GPU Clusters of Thousand-Plus Scale*. Version 1. 2025. arXiv: 2502.05413. URL: <http://arxiv.org/abs/2502.05413>. Pre-published.
- [61] *Characterization of the DOE Mini-apps*. URL: <https://portal.nersc.gov/project/CAL/doe-miniapps.htm>.
- [62] Philipp Grete et al. “Parthenon—a Performance Portable Block-Structured Adaptive Mesh Refinement Framework”. In: *The International Journal of High Performance Computing Applications* 37.5 (2023). URL: <https://doi.org/10.1177/10943420221143775>.
- [63] Weiqun Zhang et al. “AMReX: Block-structured Adaptive Mesh Refinement for Multiphysics Applications”. In: *The International Journal of High Performance Computing Applications* 35.6 (2021). URL: <https://doi.org/10.1177/10943420211022811>.
- [64] *Pandas - Python Data Analysis Library*. URL: <https://pandas.pydata.org/>.
- [65] Robert Schulze et al. “ClickHouse - Lightning Fast Analytics for Everyone”. In: *Proceedings of the VLDB Endowment* 17.12 (2024). URL: <https://dl.acm.org/doi/10.14778/3685800.3685802>.
- [66] M. S. Warren and J. K. Salmon. “A Parallel Hashed Oct-Tree N-body Algorithm”. In: *Proceedings of the 1993 ACM/IEEE Conference on Supercomputing - Supercomputing '93*. 1993. URL: <http://portal.acm.org/citation.cfm?doid=169627.169640>.
- [67] Aparna Sasidharan, John M. Dennis, and Marc Snir. “A General Space-filling Curve Algorithm for Partitioning 2D Meshes”. In: *2015 IEEE 17th International Conference on High Performance Computing and Communications, 2015 IEEE 7th International Symposium on Cyberspace Safety and Security, and 2015 IEEE 12th International Conference on Embedded Software and Systems*. 2015. URL: <https://ieeexplore.ieee.org/document/7336274/>.
- [68] Abhinav Bhatele et al. “Applying Graph Partitioning Methods in Measurement-Based Dynamic Load Balancing”. In: (2011).
- [69] Umit V. Catalyurek et al. “Hypergraph-Based Dynamic Load Balancing for Adaptive Scientific Computations”. In: *2007 IEEE International Parallel and Distributed Processing Symposium*. 2007. URL: <http://ieeexplore.ieee.org/document/4227986/>.
- [70] Dominique Lasalle and George Karypis. “Multi-Threaded Graph Partitioning”. In: *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*. 2013. URL: <https://ieeexplore.ieee.org/document/6569814>.
- [71] Carsten Burstedde, Lucas C. Wilcox, and Omar Ghattas. “P4est: Scalable Algorithms for Parallel Adaptive Mesh Refinement on Forests of Octrees”. In: *SIAM Journal on Scientific Computing* 33.3 (2011). URL: <https://epubs.siam.org/doi/10.1137/100791634>.
- [72] R. L. Graham. “Bounds on Multiprocessing Timing Anomalies”. In: *SIAM Journal on Applied Mathematics* 17.2 (1969). URL: <https://epubs.siam.org/doi/10.1137/0117039>.
- [73] Brandon Barker et al. *Phoebus: Performance Portable GRRMHD for Relativistic Astrophysics*. Version 2. 2024. arXiv: 2410.09146 [astro-ph]. URL: <http://arxiv.org/abs/2410.09146>. Pre-published.
- [74] Sedov, Leonid Ivanovich. *Similarity and Dimensional Methods in Mechanics*. 1959. URL: <http://archive.org/details/l-i-sedov.-similarity-and-dimensional-methods-in-mechanics>.
- [75] Haryadi S. Gunawi et al. “Fail-Slow at Scale: Evidence of Hardware Performance Faults in Large Production Systems”. In: *ACM Transactions on Storage* 14.3 (2018). URL: <https://dl.acm.org/doi/10.1145/3242086>.
- [76] Bilge Acun and Laxmikant V. Kale. “Mitigating Processor Variation through Dynamic Load Balancing”. In: *2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 2016. URL: <http://ieeexplore.ieee.org/document/7529982/>.
- [77] Andreas Knüpfer et al. “Score-P: A Joint Performance Measurement Run-Time Infrastructure for Periscope, Scalasca, TAU, and Vampir”. In: *Tools for High Performance Computing 2011*. Ed. by Holger Brunst et al. 2012.
- [78] *InfluxDB — Real-time Insights at Any Scale*. InfluxData. Sat, 15 Jan 2022 15:32:09 +0000. URL: <https://www.influxdata.com/home/>.
- [79] *Prometheus - Monitoring System & Time Series Database*. URL: <https://prometheus.io/>.

- [80] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019. URL: <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>.
- [81] Yuxuan Jiang et al. *Training with Confidence: Catching Silent Errors in Deep Learning Training with Automated Proactive Checks*. 2025. arXiv: 2506.14813 [cs]. URL: <http://arxiv.org/abs/2506.14813>. Pre-published.
- [82] Andrew Lamb et al. “Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine”. In: *Companion of the 2024 International Conference on Management of Data*. 2024. URL: <https://dl.acm.org/doi/10.1145/3626246.3653368>.
- [83] Jerome Soumagne et al. “Mercury: Enabling Remote Procedure Call for High-Performance Computing”. In: *2013 IEEE International Conference on Cluster Computing (CLUSTER)*. 2013. URL: <http://ieeexplore.ieee.org/document/6702617/>.
- [84] Paul Grun et al. “A Brief Introduction to the OpenFabrics Interfaces - A New Network API for Maximizing High Performance Application Efficiency”. In: *2015 IEEE 23rd Annual Symposium on High-Performance Interconnects*. 2015. URL: <http://ieeexplore.ieee.org/document/7312664/>.
- [85] *TileDB • Designed for Discovery*. TileDB. URL: <https://tiledb.com/>.
- [86] Byung S Lee and Ron Musick. “MeshSQL: The Query Language for Simulation Mesh Data”. In: *Information Sciences* 159.3 (2004). URL: <https://www.sciencedirect.com/science/article/pii/S0020025503001981>.
- [87] M. Kersten et al. “SciQL, a Query Language for Science Applications”. In: *Proceedings of the EDBT/ICDT 2011 Workshop on Array Databases*. AD ’11. 2011. URL: <https://dl.acm.org/doi/10.1145/1966895.1966896>.
- [88] Alireza Rezaei Mahdiraji. “Toward Unstructured Mesh Algebra and Query Language”. In: *Proceedings of the 2014 SIGMOD PhD Symposium*. SIGMOD’14 PhD Symposium. 2014. URL: <https://dl.acm.org/doi/10.1145/2602622.2602626>.
- [89] Jianshen Liu et al. *Processing Particle Data Flows with SmartNICs*. 2022. arXiv: 2210.06827 [cs]. URL: <http://arxiv.org/abs/2210.06827>. Pre-published.
- [90] Craig Ulmer et al. “Extending Composable Data Services into SmartNICs”. In: *2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 2023.
- [91] Mark Raasveldt and Hannes Mühleisen. “DuckDB: An Embeddable Analytical Database”. In: *Proceedings of the 2019 International Conference on Management of Data*. 2019. URL: <https://dl.acm.org/doi/10.1145/3299869.3320212>.
- [92] *Arrow Flight SQL — Apache Arrow V20.0.0*. URL: <https://arrow.apache.org/docs/format/FlightSql.html>.
- [93] *Grafana — Query, Visualize, Alerting Observability Platform*. Grafana Labs. URL: <https://grafana.com/grafana/>.
- [94] *Parquet*. Apache Parquet. URL: <https://parquet.apache.org/>.